

令和 6 年度

卒業論文

Hodgkin-Huxley モデルの計算コスト削減を
目指したサロゲートモデルの開発

指導教員 小林 泰良 助教

山口大学
理学部 物理・情報科学科
情報科学コース

棟近 春樹

目次

第 1 章	はじめに	1
第 2 章	基礎知識	2
2.1	ニューロンのモデル化	2
2.2	Hodgkin-Huxley モデル	3
第 3 章	Hodgkin-Huxley 代理モデルの作成手法	5
3.1	概要	5
3.2	シミュレーション条件	6
3.2.1	初期値の与え方	6
3.2.2	ランダムに外部電流 I_{ext} を生成する方法	7
3.3	主成分分析	8
3.4	SINDy を用いた代理モデルの ϕ, ψ の選択方法	9
3.4.1	SINDy	9
第 4 章	結果と考察	12
4.1	Hodgkin-Huxley モデルのシミュレーションと隠れ変数の圧縮	12
4.2	ゲート変数の圧縮	13
4.3	得られた Hodgkin-Huxley 代理モデル	14
4.4	代理モデルの評価	17
4.4.1	代理モデルの再現性の評価	17
4.4.2	計算コストの評価	17
4.5	考察	22
第 5 章	まとめ	23
	謝辞	24

第 1 章 はじめに

脳は生物の思考や行動などの情報処理を司り、この機能はニューロンのネットワークによるスパイクの伝播によりなされる [1]。ネットワークのスパイクを観測することで脳の情報処理過程を知ることができるが、これは脳を構成する莫大な数 (10^{11} 個程度 [2]) のニューロンすべてを同時に観測する必要があるため、非常に難しい。一方、個別のニューロンの挙動については多くのことが知られているため、ニューロンの振る舞いを記述する数理モデルを組み合わせた脳モデルを作り、シミュレーションを行うといったことも可能である。近年の研究によると、ニューロンは刺激シーケンスの弁別や排他的論理和演算などの複雑な挙動を行い、これにはニューロンそのものが持つ空間形状が大きく関わっていることが分かってきた [3][4]。ニューロンの振る舞いを記述する数理モデルである Multi-Compartment モデルはニューロンの空間形状をモデル化の際に組み入れており、このような特性を忠実に再現する。Multi-Compartment モデルを用いて脳シミュレーションを行えば、個々のニューロンの高度な演算能力がどのように脳機能を作るかを調べられる。だが、Multi-Compartment モデルはニューロンの空間形状を電氣的に連結するいくつかの区画 (Compartment) の群として表現しており、各コンパートメントは膜電位と複数の隠れ変数を状態として持つため、計算に多くのメモリを必要とする [1]。Multi-Compartment モデルを用いると、マウスの脳 (約 $10^8 \sim 10^9$ 個のニューロンを持つ) 程度のシミュレーションであっても、国内最高のスパコンである富岳でも扱いきれないほどに空間計算量が増えてしまう [5]。ここでの隠れ変数は細胞膜におけるイオン輸送を制御する重要な変数であるが、脳シミュレーションにおいてはこの値を直接的に知る必要はない。にもかかわらず、隠れ変数を格納する上で必要となるメモリの量はニューロン数とコンパートメント数に比例して大きくなってしまふ。大規模な脳モデルのシミュレーションを行うには、ニューロンモデルをシミュレーションする際の空間計算量を削減することが重要である。

ニューロンモデルの空間計算量を削減する手法として、代理モデル (surrogate model) の作成を考える。ここでの代理モデルは、特定の Multi-Compartment モデルに対し、より少ない数の隠れ変数で同等の膜電位応答を再現することができるモデルとする。Multi-Compartment モデルはニューロンモデルの中でも空間形状を考慮した複雑なモデルであるため、いきなりこれに対する代理モデルを作成するのは難易度が高い。Multi-Compartment モデルより基本的なニューロンモデルで代理モデルが機能するのかをまずは確かめたい。そのようなモデルとして、Hodgkin-Huxley モデルを選ぶ [6]。Hodgkin-Huxley モデルは、空間形状を考慮しない単純なモデルであるが、Multi-Compartment モデル同じようにイオン輸送を制御する隠れ変数を持つ。まとめると本研究では、Hodgkin-Huxley モデルに対する代理モデルの構築を行う。

第 2 章 基礎知識

2.1 ニューロンのモデル化

脳は聴覚や視覚などの感覚情報の処理、記憶、思考、筋肉への運動司令などを司る情報処理器官であり、これはニューロンのネットワークによって実現される。ニューロンは電流の入力を受けると、細胞の内外の電位差である膜電位を変化させる。膜電位の変化には、細胞膜と、細胞膜に埋め込まれたイオンチャネル (特定のイオンのみを選択的に透過させる輸送体) が深く関わっている。ニューロンの電氣的な振る舞いは、図 2.1 のように、細胞膜を容量 C [$\mu\text{F}/\text{cm}^2$] のキャパシタ、膜を透過するイオンの流れ I_{leak} [$\mu\text{A}/\text{cm}^2$] を内部コンダクタンス g_{leak} [mS/cm^2] である起電力 E_{eq} [mV] の電池を流れる電流と見立てることで、RC 回路とみなすことが可能である [7]。この回路モデルでは、外部から入力される電流を I_{ext} [$\mu\text{A}/\text{cm}^2$] と記述している。そして、この回路モデルは RC 回路であるため、膜電位 V の時間変化は式 (2.1) の微分方程式に従う。

$$C \frac{dV}{dt} = -g_{\text{leak}}(V(t) - E_{\text{eq}}) + I_{\text{ext}}(t) \quad (2.1)$$

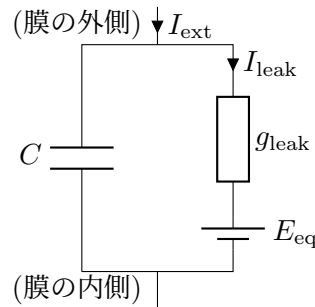


図 2.1: 細胞膜に対応する RC 回路。[1] より改変。

このモデルはニューロンの細胞膜についての基本的なダイナミクスを記述している。だが、これだけでは単に膜電位が上下するだけでスパイクは発射されない。後述の Hodgkin-Huxley モデルのように、イオンチャネルの種類まで考慮に入れてモデル化することで、ニューロンのスパイクの発射を再現することが可能となる。

2.2 Hodgkin-Huxley モデル

単一ニューロンの数理モデルの中で世界で最初に開発されたものが Hodgkin-Huxley モデルである [1]。Hodgkin-Huxley モデルはイオン電流をナトリウムイオン電流 I_{Na} [$\mu\text{A}/\text{cm}^2$]、カリウムイオン電流 I_{K} [$\mu\text{A}/\text{cm}^2$]、リーク電流 I_{leak} [$\mu\text{A}/\text{cm}^2$](その他のイオン電流) に分離させたモデルである。Hodgkin-Huxley モデルでは、ニューロンの電氣的な振る舞いを図 2.2a のような回路モデルと見立てることができる [7]。この回路モデルは、入力電流を I_{ext} [$\mu\text{A}/\text{cm}^2$]、細胞膜を容量 $C(=1.00 \mu\text{F}/\text{cm}^2)$ のキャパシタ、膜を透過するナトリウムイオン、カリウムイオン、その他のイオンの流れを内部コンダクタンスが g_{Na} [mS/cm^2], g_{K} [mS/cm^2], g_{leak} [mS/cm^2] の起電力が E_{Na} [mV], E_{K} [mV], E_{eq} [mV] の電流 I_{Na} 、 I_{K} 、 I_{leak} と見立てている。

膜電位 V [mV] についての変化式は、式 (2.1) のイオン電流を記述する項を分割することで、以下のように書き表せる。

$$C \frac{dV}{dt} = -g_{\text{leak}}(V(t) - E_{\text{leak}}) - g_{\text{Na}}(V, t)(V(t) - E_{\text{Na}}) - g_{\text{K}}(V, t)(V(t) - E_{\text{K}}) + I_{\text{ext}}(t) \quad (2.2)$$

ここで、ナトリウムコンダクタンス $g_{\text{Na}}(V, t)$ とカリウムコンダクタンス $g_{\text{K}}(V, t)$ は V, t に従って変化する。

$$g_{\text{Na}}(V, t) = \bar{g}_{\text{Na}} m^3(V, t) h(V, t) \quad (2.3)$$

$$g_{\text{K}}(V, t) = \bar{g}_{\text{K}} n^4(V, t) \quad (2.4)$$

$\bar{g}_{\text{Na}}$, $\bar{g}_{\text{K}}$ は各チャネルのコンダクタンス $g_{\text{Na}}(V, t)$, $g_{\text{K}}(V, t)$ がとりうる最大値を表す定数 (表 2.1) であり、 m, h, n は膜電位に依存して変化するゲート変数である。ゲート変数はイオンチャネルの開閉率を表す 0~1 の間の値を取る変数であり、膜電位と同様に、次の時間に関する一階の微分方程式に従う。

$$\begin{aligned} \frac{d}{dt} m(V, t) &= \alpha_m(V)(1 - m(V, t)) - \beta_m(V)m(V, t) \\ \frac{d}{dt} h(V, t) &= \alpha_h(V)(1 - h(V, t)) - \beta_h(V)h(V, t) \\ \frac{d}{dt} n(V, t) &= \alpha_n(V)(1 - n(V, t)) - \beta_n(V)n(V, t) \end{aligned} \quad (2.5)$$

$\alpha_g(V)$, $\beta_g(V)$ ($g = m, h, n$) はそれぞれ次式で定義される。

$$\begin{aligned} \alpha_m(V) &= \frac{2.5 - 0.1V}{\exp(2.5 - 0.1V) - 1} & \beta_m(V) &= 4 \exp(-\frac{V}{18}) \\ \alpha_h(V) &= 0.07 \exp(-\frac{V}{20}) & \beta_h(V) &= \frac{1}{\exp(3 - 0.1V) - 1} \\ \alpha_n(V) &= \frac{0.1 - 0.001V}{\exp(1 - 0.1V) - 1} & \beta_n(V) &= 0.125 \exp(-\frac{V}{80}) \end{aligned}$$

まとめると、Hodgkin-Huxley モデルは、ゲート変数 m, h, n と膜電位 V に関する 4 元の連立常微分方程式である。

表 2.1: Hodgkin-Huxley モデルで使うイオンチャンネルに関する定数

イオン種	反転電位 E [mV]	最大コンダクタンス $\bar{g}$ [S]
Na^+	115.0	120.0
K^+	-12.0	36.0
その他のイオン (<i>leak</i>)	10.6	0.3

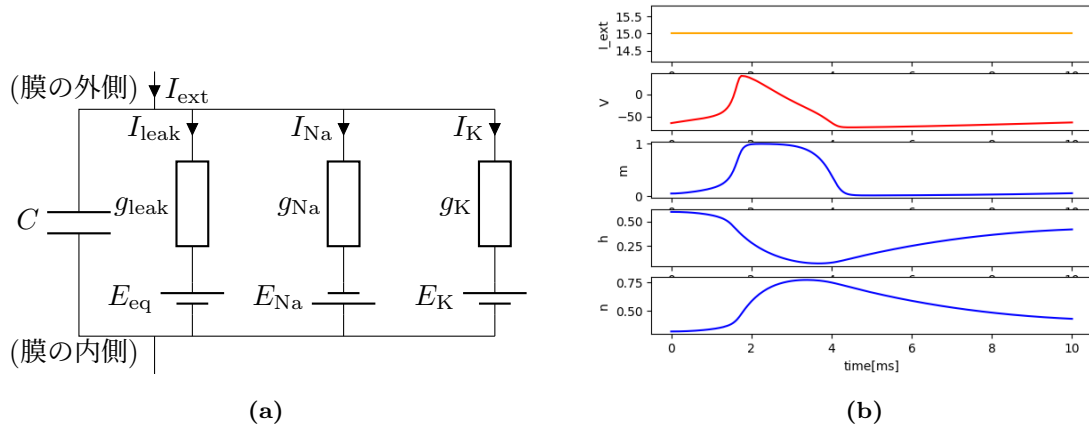


図 2.2: Hodgkin-Huxley モデル。(a)Hodgkin-Huxley モデルの回路図 (b) シミュレーションによる Hodgkin-Huxley モデルの挙動。15 mA の電流を 10 ms 与えた場合。図は上から膜電位 V , ゲート変数 m, h, n の時間変化を表す

第 3 章 Hodgkin-Huxley 代理モデルの作成手法

3.1 概要

Hodgkin-Huxley モデルは、外部電流 $I_{\text{ext}}(t)$ に対する膜電位 $V(t)$ の時間変化を $V(t)$ と 3 つのゲート変数 (隠れ変数) である $m(t), n(t), h(t)$ から成る 4 元の連立常微分方程式で記述したモデルである (2.2 節)。本研究では、外部電流に対する膜電位 $V(t)$ の時間変化をより少ないメモリ使用量でシミュレーション可能な代理モデルの作成を目指す。代理モデルとして、膜電位 $V(t)$ の時間変化を $V(t)$ と 1 つの隠れ変数 $g'(t)$ の 2 つのみからなる式 (3.1) のような二元の連立方程式で記述可能なモデルを作成する。このような代理モデルは、Hodgkin-Huxley モデルより隠れ変数の数を少ないため、メモリ使用量が少ない。

$$\frac{dV}{dt} = \phi(V, g', I_{\text{ext}}), \frac{dg'}{dt} = \psi(V, g', I_{\text{ext}}) \quad (3.1)$$

具体的な手法としては、はじめに、Hodgkin-Huxley モデルのシミュレーションを陽的 Euler 法を用いて行い、ランダムな外部入力 I_{ext} に対する $v(t), m(t), n(t), h(t)$ の時間変化の時系列データを取得する (3.2 節)。次に、隠れ変数の数を 1 つにするため、 $m(t), n(t), h(t)$ の時系列データに対し主成分分析 (3.3 節) を施すことで、1 次元に圧縮した隠れ変数の時系列データを取得する。この時系列データをあらたな隠れ変数 $g'(t)$ のデータとする。最後に、外部電流 $I_{\text{ext}}(t)$ を与えた場合、 $V(t)$ と $g'(t)$ の時系列データを再現 (シミュレーション) 可能な二元の連立方程式 (代理モデル) を作成する。ここで、代理モデルを作成するための手法として SINDy を使用する (3.4 節)。SINDy とは、時系列データが与えられた場合、事前に与えた基底関数からこのデータを説明可能な重みの組み合わせを推定する手法である。このようにして隠れ変数の数を削減し、計算コストを抑えた Hodgkin-Huxley モデルの代理モデルを作成する。

3.2 シミュレーション条件

Hodgkin-Huxley モデルや代理モデルの実体は下式のような n 元 1 階連立常微分方程式である。

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t)$$

$$\mathbf{x}(t) = \begin{pmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_n(t) \end{pmatrix} \quad \mathbf{f}(t, \mathbf{x}) = \begin{pmatrix} f_1(t, x_1, x_2, \dots, x_n) \\ f_2(t, x_1, x_2, \dots, x_n) \\ \vdots \\ f_n(t, x_1, x_2, \dots, x_n) \end{pmatrix}$$

今回扱うモデルにおいては、 $f_1, f_2, \dots, f_n$ は非線形の関数であるため、その解析解を求めることは非常に困難である。そこで、モデルの挙動のシミュレーションの手法として、数値計算手法の一つである Euler 法を使った。Euler 法を使えば、区間 $[a, b]$ を長さ h で等間隔に n 等分した点 $t_0, t_1, \dots, t_n$ に対し、各点での近似解 $x_0, x_1, \dots, x_n$ を求めることができる。簡単な 1 変数の場合 ($\frac{dx}{dt} = f(x, t)$) のアルゴリズムを Algorithm 1 に示す。これは式の形が同じであるため、容易に多変数の場合 ($\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t)$) にも拡張可能である。

Algorithm 1 Euler 法 [8] より改変

Require: 初期値: c_0 , 刻み幅: dt

Ensure:

```

 $x_0 \leftarrow c_0$ 
for  $i = 0, k \leq n, i++$  do
     $x_{i+1} = x_i + hf(x_i, t_i)$ 
end for
return  $x_0, x_1, \dots, x_n$ 

```

次にこのアルゴリズムの妥当性を考える。一般に、 $\frac{dx}{dt} = f(x, t)$ の解と成る関数 $x = g(t)$ は $t=a$ 付近で 1 次の Taylor 展開を行えば、

$$x = g(a) + g'(a)(t - a) + o((t - a)^2) \quad (t \leftarrow a) \quad (3.2)$$

が成立する。この式を用い、 $t \leftarrow t_{i+1}, a \leftarrow t_i$ として、 $g(t_i) = x_i$ から $g(t_{i+1}) = x_{i+1}$ を推定すること考える。すると、

$$x_{i+1} = x_i + hg'(t_i) \quad (3.3)$$

が得られる。これは Euler 法のアルゴリズムと完全に一致する。

3.2.1 初期値の与え方

微分方程式を数值的又は解析的に解くためには、初期値を与えることが必要である。Hodgkin-Huxley モデル、代理モデルいずれにおいても、膜電位の初期値は定数 $v_{rest} = -65.0$ を与える。

ゲート変数の初期値

Hodgkin-Huxley モデルのゲート変数の式は、

$$\frac{dg}{dt} = \alpha_g(V)(1 - g) - \beta_g(V)g \quad (g = m, h, n) \quad (3.4)$$

で表すことができる。これを変形すると、

$$\frac{1}{\alpha_g(V) + \beta_g(V)} \frac{dg}{dt} = -\left(g - \frac{\alpha_g(V)}{\alpha_g(V) + \beta_g(V)}\right) \quad (3.5)$$

となる。この式は、ゲート変数 g は $\alpha_g(V)/(\alpha_g(V) + \beta_g(V))$ において $\frac{dg}{dt} = 0$ で定常状態になることを意味する。そして、ゲート変数は時刻 $t=0$ の初期値において傾きが 0 であってほしい。そこで、Hodgkin-Huxley モデルのゲート変数の初期値は、

$$\begin{pmatrix} m_0 \\ h_0 \\ n_0 \end{pmatrix} = \begin{pmatrix} \alpha_m(V_0)/(\alpha_m(V_0) + \beta_m(V_0)) \\ \alpha_h(V_0)/(\alpha_h(V_0) + \beta_h(V_0)) \\ \alpha_n(V_0)/(\alpha_n(V_0) + \beta_n(V_0)) \end{pmatrix} \quad (3.6)$$

となる。代理モデルのゲート変数 g' の初期値は、Hodgkin-Huxley モデルの初期値 (m_0, h_0, n_0) を教師データと同様に主成分分析で圧縮させた値とする (3.3 節)。

3.2.2 ランダムに外部電流 I_{ext} を生成する方法

教師データには、なるべく様々な入力とそれに対する応答が存在することが望ましい。様々な入力が教師データの中に存在すれば、特定の入力に依存することがないロバストなモデルの作成が期待できる。そこで教師データに用いる外部電流 I_{ext} はランダムな複数の定常電流の組み合わせとして与えた。具体的には、Algorithm 2 のように与えた。

Algorithm 2 I_{ext} の決定

Require: シミュレーション時間: t , 時間間隔: $\Delta (< t)$, 最大電流: $\max$

Ensure:

シミュレーションの時間 $[0, t]$ を Δ で分割し、 $t_0 = [0, \Delta], t_1 = [\Delta, 2 \cdot \Delta], \dots, t_n = [t_n \cdot \Delta, t]$ を得る

for $i \in [0, 1, \dots, n]$ **do**

$0 \sim \max$ の中からランダムに数 rnd を一つ選ぶ

t_i の間には $rnd [\text{mA}]$ の定常電流 $I_{\text{ext}(i)}$ が流れているものとする

end for

return $I_{\text{ext}} = [I_{\text{ext}(0)}, I_{\text{ext}(1)}, \dots, I_{\text{ext}(n)},]$

3.3 主成分分析

本研究で主成分分析の対象とするゲート変数 m, h, n のデータには、図 2.2b をみてもわかるよう、変数間に相関性がある。すなわち、図 3.1 のように、3 次元空間内に均等に散布されていない。このような場合には、データがもつ本質的な情報はより少ない成分でも表現できると期待できる。一般的に言うと、データが高次元中の平坦な部分空間に偏在している場合、データが広がっている方向を考えることで、より少ない次元でデータを扱えることが見込める。このような解析を一般に行うのが、主成分分析 (Principal Component Analysis, PCA) である。

主成分分析を行うためには、解析の対象と成るデータが必要である。データ点は D 次元のベクトル

$$\mathbf{x} = (x_1, x_2, \dots, x_D)^T \quad (3.7)$$

で表すとする。

そして、このようなデータ点は N 個存在し、すべてのデータは、 $X = (\mathbf{x}_1, \dots, \mathbf{x}_N) \in \mathbf{R}^{N \times D}$ で表現できるとする。次に、これらのデータ点について、ばらつきの大きい順に適当な $d (\leq D)$ 個の基底ベクトル

$$\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_d \quad (3.8)$$

を考える。これらの基底ベクトルで貼ることのできる部分空間にだいたいのデータ点が存在していると仮定する。ある点 $\mathbf{c}_0$ を中心点として、一般のデータ点をこの部分空間に射影してみる。

$$\mathbf{x}_n \approx \mathbf{c}_0 + \sum_{h=1}^d \left(\mathbf{e}_h \cdot (\mathbf{x}_n - \mathbf{c}_0) \right) \mathbf{e}_h \quad (3.9)$$

この射影が元のデータ点をどれだけうまく表しているかの測定には、下式 (3.10) のような二乗誤差関数を使えば良い。

$$E(\mathbf{c}_0, \mathbf{e}_h) = \sum_{n=1}^N \left((\mathbf{x}_n - \mathbf{c}_0) - \sum_{h=1}^d (\mathbf{e}_h \cdot (\mathbf{x}_n - \mathbf{c}_0)) \mathbf{e}_h \right) \quad (3.10)$$

主成分分析とはすなわち、式 (3.10) の最小化問題である。

これを解くと、 $\mathbf{c}_0 = \sum_n \mathbf{x}_n / N$ となる [9]。そして、 $\mathbf{e}_h$ はデータ X を共分散行列 $\Phi_{ij} = \sum_n (x_{ni} - \bar{x}_i)(x_{nj} - \bar{x}_j)$ として、

$$\Phi \mathbf{e}_h = \lambda_h \mathbf{e}_h \quad (3.11)$$

を満たすを満たすことが求められる。すなわち最大化問題は、共分散行列 Φ の固有ベクトルを求めることと同値である。 Φ の固有ベクトルそのものは D 個存在するが、このうち解と成るものは、固有値を大きい順に d 個並べた $\lambda_0, \lambda_1, \dots, \lambda_d$ に対応する固有ベクトルである。

実際に行った操作としては、データ X の共分散行列 Π と、各データ点について $\mathbf{x}'_n \leftarrow \mathbf{x}_n - \mathbf{c}_0$ で正規化させた X' を計算する。次に、 Φ の固有ベクトルを固有値が大きい順に d 個並べ、行列 $\Gamma = (\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_d)$ を得る。行列積 $X'\Gamma$ が実際に得られる圧縮されたデータである。

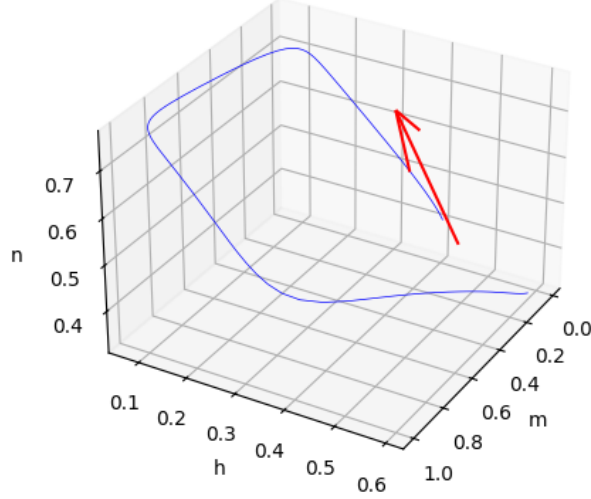


図 3.1: 図 2.2b のゲート変数の軌跡。赤: ゲート変数を主成分分析で圧縮する際の射影軸

3.4 SINDy を用いた代理モデルの ϕ, ψ の選択方法

下式の関数 $\theta_0, \theta_1 \dots$ と定数 a_{ij} を適切に選んだものの代理モデルとする

$$\begin{aligned}\frac{d}{dt}V &= \phi(V, g', I_{\text{ext}}) = a_{11}\theta_0(V, g', I_{\text{ext}}) + a_{12}\theta_1(V, g', I_{\text{ext}}) + \dots \\ \frac{d}{dt}g' &= \psi(V, g', I_{\text{ext}}) = a_{21}\theta_0(V, g', I_{\text{ext}}) + a_{22}\theta_1(V, g', I_{\text{ext}}) + \dots\end{aligned}$$

θ_0, θ_1 をハイパーパラメーターとして事前に与えられる。そして、入力 I_{ext} に対し、時系列データ V, g' を再現するように a_{ij} をフィッティングさせたこれは下記の SINDy における STLSQoptimizer[10] で達成できる。

3.4.1 SINDy

SINDy(非線形ダイナミクスのスパース同定: Sparse Identification of Nonlinear Dynamics の略)[11] とは、Brunton によって定式化された機械学習モデルである。SINDy モデルを利用することにより、時系列データが与えられたとき、これを説明できるような一階の微分方程式の構築ができる。具体的には

$$\frac{d}{dt}\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t)) \quad (3.12)$$

を満たす関数 f の推定ができる。実際に観測されたデータを

$$X = \begin{pmatrix} x_1(t_1) & x_2(t_1) & \cdots & x_n(t_1) \\ x_1(t_2) & x_2(t_2) & \cdots & x_n(t_2) \\ \vdots & \vdots & \ddots & \vdots \\ x_1(t_m) & x_2(t_m) & \cdots & x_n(t_m) \end{pmatrix} = \begin{pmatrix} \mathbf{x}(t_1) \\ \mathbf{x}(t_2) \\ \vdots \\ \mathbf{x}(t_m) \end{pmatrix} \quad (3.13)$$

とする。SINDy を適用させるには、まずはじめに、観測された時系列データ X から以下の式で表せる速度 $\dot{X}$ を計算する。

$$\dot{X} = \begin{pmatrix} \dot{x}_1(t_1) & \dot{x}_2(t_1) & \cdots & \dot{x}_n(t_1) \\ \dot{x}_1(t_2) & \dot{x}_2(t_2) & \cdots & \dot{x}_n(t_2) \\ \vdots & \vdots & \ddots & \vdots \\ \dot{x}_1(t_m) & \dot{x}_2(t_m) & \cdots & \dot{x}_n(t_m) \end{pmatrix} \quad (3.14)$$

次に、 p 個 ($p \ll m$) の基底関数 $\theta_i(x_1, \dots, x_n)$ をデータに応じて適切に選択する。ここで、与えられた基底関数と X から、下式で定義された行列 $\Theta(X)$ を計算する。

$$\Theta(X) = \begin{pmatrix} \theta_1(\mathbf{x}(t_1)) & \theta_2(\mathbf{x}(t_1)) & \cdots & \theta_p(\mathbf{x}(t_1)) \\ \theta_1(\mathbf{x}(t_2)) & \theta_2(\mathbf{x}(t_2)) & \cdots & \theta_p(\mathbf{x}(t_2)) \\ \vdots & \vdots & \ddots & \vdots \\ \theta_1(\mathbf{x}(t_m)) & \theta_2(\mathbf{x}(t_m)) & \cdots & \theta_p(\mathbf{x}(t_m)) \end{pmatrix} \quad (3.15)$$

$\dot{X}$ と $\Theta(X)$ は下式のように p 行 m 列の係数行列 $\Xi = a_{ij}$ で結び付けられる。

$$\dot{X} = \Theta(X)\Xi \quad (3.16)$$

この係数行列 Ξ を等式を満たすようにフィッティングするのが SINDy である。SINDy の実装には、Python 上で作られた pySINDy ライブラリ [12][13] を利用した。

STLSQ Optimizer

SINDy のフィッティングのためのオプティマイザーには STLSQ[10] を選んだ。STLSQ は、逐次閾値最小二乗アルゴリズム (sequentially Thresholded Least-Squares Algorithm) の略であり、これは SINDy を提唱した Brunton が使った手法である。このオプティマイザーは、行列 $X \in \mathbb{R}^{m \times p}$ 、 $Y \in \mathbb{R}^{m \times n}$ が与えられたとき、

$$Y = XB \quad (3.17)$$

を満たすようなスパースな行列 $B \in \mathbb{R}^{p \times n}$ の回帰問題である。SINDy においては、式 (3.16) の $\dot{X}$ に Y 、 $\Theta(X)$ に X 、 Ξ に B が対応する。STLSQ の具体的なアルゴリズムは Algorithm 3 である。SINDy を適用させることによって得られた $B^k = \Xi$ はスパースな行列となる。

Algorithm 3 STLSQ algorithm [14] より改変

Require: 閾値 $\lambda \geq 0, X \in \mathbb{R}^{m \times p}, Y \in \mathbb{R}^{m \times n}$

Ensure:

```
 $B^0 \leftarrow$  式  $Y = X B^0$  に対する最小二乗法で得られる擬逆行列  
 $S^0 \leftarrow$  行列  $B^0$  の絶対値が閾値  $\lambda$  以上の要素に対するインデックス行列  
 $k=0$   
while  $S^k \neq S^{k-1}$  do  
  for  $j \in [0, 1, \dots, n]$  do  
     $\text{ind} \leftarrow S^k$  の  $j$  列目のインデックス集合  
     $X' \leftarrow X$  から  $\text{ind}$  の該当列だけ抜き出したもの  
     $Y' \leftarrow Y$  の  $j$  列目  
     $B_{k+1} \in \mathbb{R}^{p \times n}$  を零行列として初期化  
     $B_{j, \text{ind}}^{k+1} \leftarrow$  式  $Y' = X' B_{j, \text{ind}}^{k+1}$  に対する最小二乗法で得られるベクトル  
     $B^{k+1}$  の  $\text{ind}$  行  $j$  列目の要素に  $B_{j, \text{ind}}^{k+1}$  を代入  
  end for  
   $S^{k+1} \leftarrow$  行列  $B^{k+1}$  の絶対値が閾値  $\lambda$  以上の要素に対するインデックス行列  
   $k=k+1$   
end while  
return  $B^k$ 
```

ハイパラメーター $\theta_0, \theta_1 \dots$ の選択方法

まず、Hodgkin-Huxley モデルの右辺を線形和に分解する。例えば、式 (2.5) では、その右辺を分解すると

$$\alpha_m(V)(1 - m) - \beta_m(V)m = \alpha_m(V) - \alpha_m(V)m - \beta_m(V)m$$

となる。分解された時に出てきた各項の関数 $\alpha_m(V), \alpha_m(V)m, \beta_m(V)m$ に対し、変数 m, h, n を g' に置き換えた関数群 $\alpha_m(V), \alpha_m(V)g', \beta_m(V)g'$ を考える。これらに対し、適当な定数を掛けた $10^6 \cdot \alpha_m(V), 10^1 3\alpha_m(V)m, 10^1 0\beta_m(V)m$ 等を基底関数とした。(使用した基底関数の一覧は Appendix に)

STLSQoptimizer の下では、適当な定数を掛けることで、代理モデルの右辺の項がスパースになる。これは疑似コードを見ればわかるとおり、STLSQoptimizer は閾値以下の係数をカッティングを繰り返す操作を行うことに由来する。定数は、代理モデルが適切に機能する範囲で許容される最も大きい値を選んだ。

第 4 章 結果と考察

4.1 Hodgkin-Huxley モデルのシミュレーションと隠れ変数の圧縮

代理モデルの構築を行うために、まずはランダムに電流 I_{ext} を生成した。電流 I_{ext} の生成にはパラメータとして、電流のとりうる最大値には 20 mA、定常電流として与える間隔 Δ には 10 ms を用いた。次に、Hodgkin-Huxley モデルの入力電流を I_{ext} として、陽的 Euler 法によるシミュレーションをタイムステップ dt を 0.05 ms として 900 ms 間実行した。この時の Hodgkin-Huxley モデルに対しての入力電流と、その応答を図 4.1 に示す。図 4.1 の応答は、閾値を上回ればスパイクを発射するといったニューロンの持つ性質を反映している。

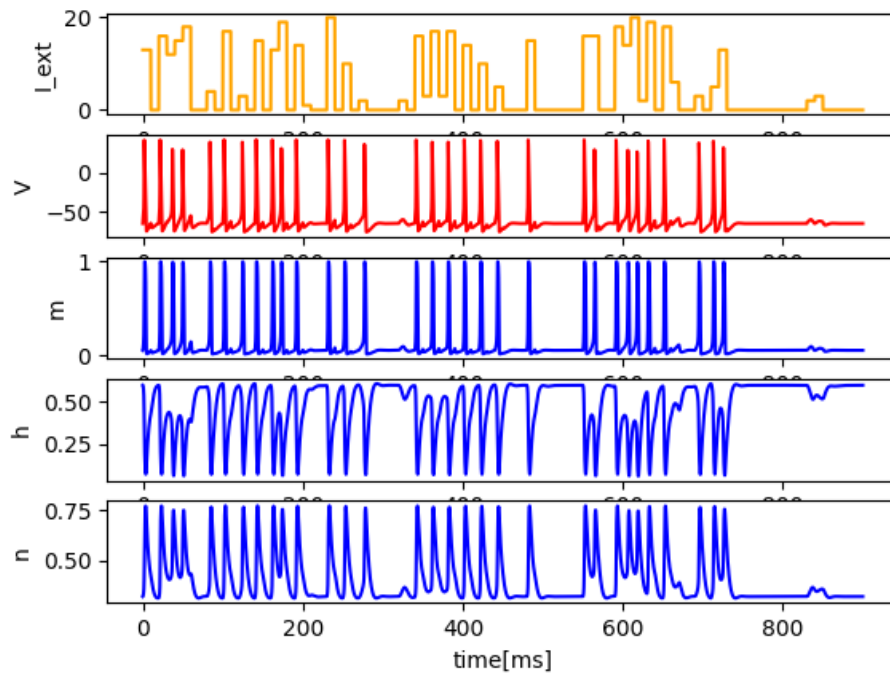


図 4.1: 入力電流 I_{ext} と、 I_{ext} 加えながらシミュレーションをした結果。黄: 入力電流, 赤: 膜電位 V の応答, 青: ゲート変数 m, h, n の応答。

4.2 ゲート変数の圧縮

図 4.1 のゲート変数 m, h, n のデータに対し、主成分分析を行った。3 つの変数 m, h, n の時系列データを主成分分析によって得られた新たな軸に射影することによって、圧縮を行った (図 4.2)。そして、その時の寄与率は 0.79 あった。ゲート変数が新たな軸によって圧縮された結果 g' は、図 4.3 に示す。入力 I_{ext} に対して、図 4.3 のように時系列データ V, g' を再現する代理モデルを構築する。

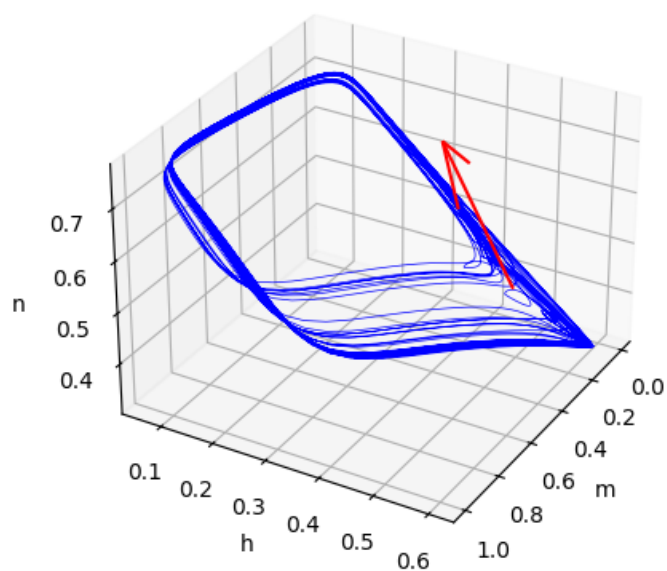


図 4.2: 図 4.1 に示したゲート変数 m, h, n の軌跡。赤い矢印: ゲート変数を主成分分析で圧縮する際の射影軸

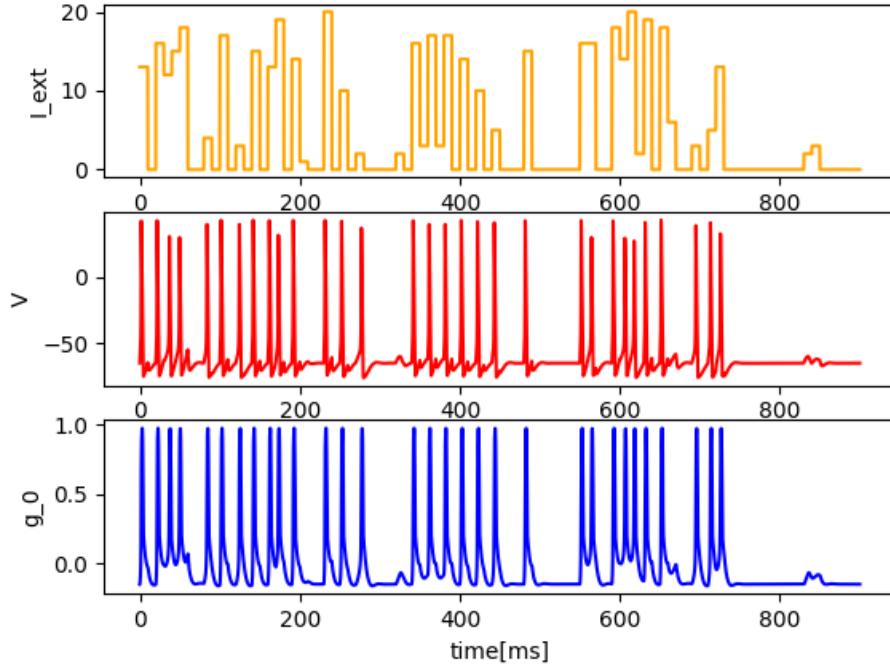


図 4.3: 代理モデルに求める振る舞い。 I_{ext} , V は図 4.1 と同じ。 g_0 は図 4.1 のゲート変数のデータに対し主成分分析を施したもの。

4.3 得られた Hodgkin-Huxley 代理モデル

図 4.3 のデータを再現するように、3.4 節に述べた手法で代理モデルを構築した。モデル構築の際に使用した SINDy の基底関数 $\theta_i(V, g', I_{\text{ext}})$ を表 4.1 に示す。これら基底関数を使用して得られた代理モデルは、式 (4.1) である。

$$\begin{aligned} \frac{dV}{dt} &= \sum_{i=1}^{41} a_i \theta_i(V, g', I_{\text{ext}}) \\ \frac{dg'}{dt} &= \sum_{i=1}^{41} b_i \theta_i(V, g', I_{\text{ext}}) \end{aligned} \quad (4.1)$$

フィッティングによって得られた、基底関数 $\theta_i(V, g', I_{\text{ext}})$ に対する係数 a_i, b_i もまた表 4.1 に示す。 $a_i, b_i (i = 1 \dots 41)$ の計 82 個の係数の内、38 個が 0 であった。基底関数に定数を掛けることで最適化を行う前は 0 となる係数の数 17 個であった。このことから、係数をよりスパースにすることによって時間計算量を削減する試みはある程度機能しているといえる。

表 4.1: Hodgkin-Huxley 代理モデル作成に使用した基底関数と基底関数に対して得られた係数

Index (i)	使用した基底関数 (θ_i)	V に関する係数 (a_i)	g' に関する係数 (b_i)
1	$10^2 \cdot \alpha_m(V) \cdot g'$	7.2884069606015505	-0.027883356524977733
2	$10^2 \cdot \alpha_m(V) \cdot I_{\text{ext}}$	0.014872507899028373	0.0
3	$10^2 \cdot \alpha_m(g') \cdot I_{\text{ext}}$	-4.187859729686729	0.0
4	$10 \cdot \beta_m(V) \cdot g'$	2.29018336870304	0.0
5	$10 \cdot \beta_m(V) \cdot I_{\text{ext}}$	-0.0039786797871231286	0.0
6	$10 \cdot \beta_m(g') \cdot I_{\text{ext}}$	-44.89684639459913	0.0
7	$10^2 \cdot \alpha_h(V) \cdot g'$	-22.99445692815168	0.0
8	$10^2 \cdot \alpha_h(V) \cdot I_{\text{ext}}$	0.040119461880567496	0.0
9	$10^2 \cdot \alpha_h(g') \cdot I_{\text{ext}}$	269.5905217331277	0.0
10	$1 \cdot \beta_h(V) \cdot g'$	348.6133558206322	0.0
11	$1 \cdot \beta_h(V) \cdot I_{\text{ext}}$	-2.124402437918573	0.0
12	$1 \cdot \beta_h(g') \cdot I_{\text{ext}}$	0.0	0.0
13	$1 \cdot \alpha_n(V) \cdot g'$	-6231.649663668909	0.0
14	$1 \cdot \alpha_n(V) \cdot I_{\text{ext}}$	-0.11151823127898307	-0.06471796344111402
15	$1 \cdot \alpha_n(g') \cdot I_{\text{ext}}$	0.0	0.0
16	$10^5 \cdot \beta_n(V) \cdot g'$	0.05017027158710119	0.0
17	$10^5 \cdot \beta_n(V) \cdot I_{\text{ext}}$	0.0	0.0
18	$10^5 \cdot \beta_n(g') \cdot I_{\text{ext}}$	0.0	0.0
19	$1 \cdot \alpha_m(V)$	-952.3681339131996	3.909584075044857
20	$1 \cdot \alpha_m(g')$	7081800.160744289	-2445.8897742685945
21	$1 \cdot \alpha_m(I_{\text{ext}})$	29.88758970014755	0.032953089777687364
22	$1 \cdot \alpha_h(V)$	64.63032521783468	0.5028849825850708
23	$1 \cdot \alpha_h(g')$	-31497189.354698922	-10523.265489760686
24	$1 \cdot \alpha_h(I_{\text{ext}})$	-428.89654050886264	-1.2882166226550875
25	$1 \cdot \alpha_n(V)$	5932.956847089296	-18.09041982822185
26	$1 \cdot \alpha_n(g')$	-7382875.845970013	0.0
27	$1 \cdot \alpha_n(I_{\text{ext}})$	-1.1777271930062345	0.0315993060565311
28	$10^{10} \cdot V \cdot V \cdot V \cdot g' \cdot I_{\text{ext}}$	0.0	0.0
29	$1 \cdot V \cdot V \cdot V \cdot g'$	0.0	0.0
30	$1 \cdot V \cdot V \cdot V \cdot I_{\text{ext}}$	0.0	0.0
31	$1 \cdot g' \cdot g' \cdot g' \cdot I_{\text{ext}}$	2.6897818209468327	0.18522474276732367
32	$1 \cdot V \cdot V \cdot V \cdot V \cdot g'$	0.0	0.0
33	$1 \cdot V \cdot V \cdot V \cdot V \cdot I_{\text{ext}}$	0.0	0.0
34	$1 \cdot g' \cdot g' \cdot g' \cdot g' \cdot I_{\text{ext}}$	-0.6567249656271313	-0.1973048239113434

Index (i)	使用した基底関数 (θ_i)	V に関する係数 (a_i)	g' に関する係数 (b_i)
35	$1 \cdot V \cdot V \cdot V \cdot V$	0.0	0.0
36	$1 \cdot g' \cdot g' \cdot g' \cdot g'$	-166.08109269861941	1.7093133018920268
37	$1 \cdot I_{\text{ext}} \cdot I_{\text{ext}} \cdot I_{\text{ext}} \cdot I_{\text{ext}}$	0.0	0.0
38	1	1051264.861182022	1285.8490357190685
39	V	5.920337006209476	0.05106396400708375
40	g'	-195174.2302419484	0.0
41	I_{ext}	0.0	0.0

4.4 代理モデルの評価

4.4.1 代理モデルの再現性の評価

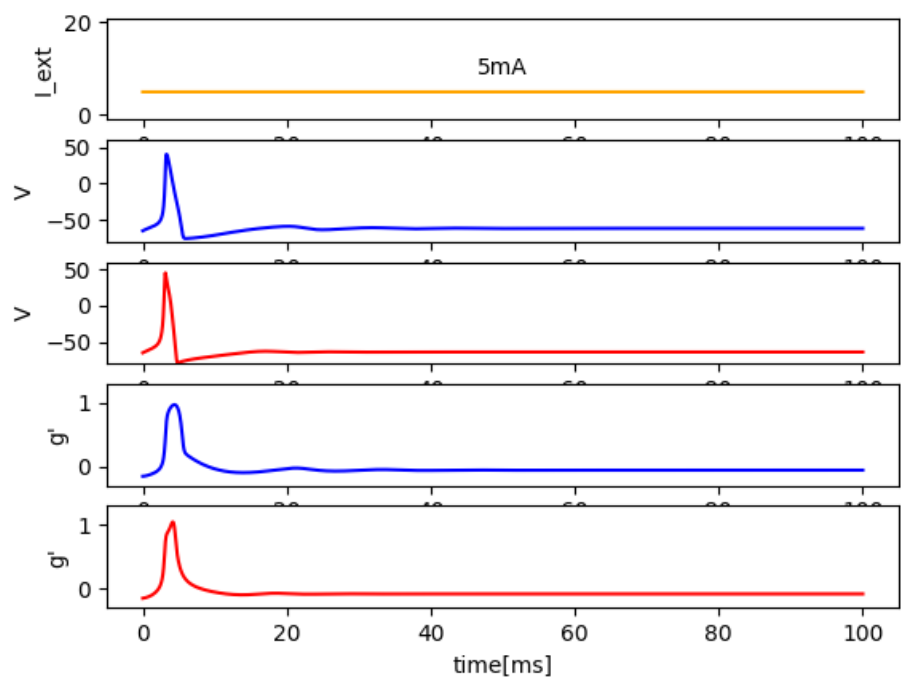
式 (2.2) , (2.5) で表される Hodgkin-Huxley モデルと式 (4.1) で表される代理モデルの各モデルに対し、それぞれ同じ電流を与え、その時の応答を比較した。各モデルには、5 mA, 10 mA, 15 mA の定常電流と、図 2.2b での教師データに用いたのと同じ電流を与えた。それぞれの結果を、図 4.4、図 4.5、図 4.6、図 4.7 に示す。いずれの場合にでも、b 図の軌跡から、大まかなダイナミクスは再現できているが、完全に一致はしなかった。図 4.4a、図 4.5a、図 4.6a を見ると、定常な電流を与えた場合、代理モデルは元のモデルとスパイクを発射するタイミングも等しく、同じような応答を見せた。また、図 4.7a の 0 ~ 100 ms 間など、定常ではない電流を与えた場合、スパイクのタイミングは必ずしも一致しなかったが、代理モデルは Hodgkin-Huxley モデルの膜電位応答をおおむね再現できた。

4.4.2 計算コストの評価

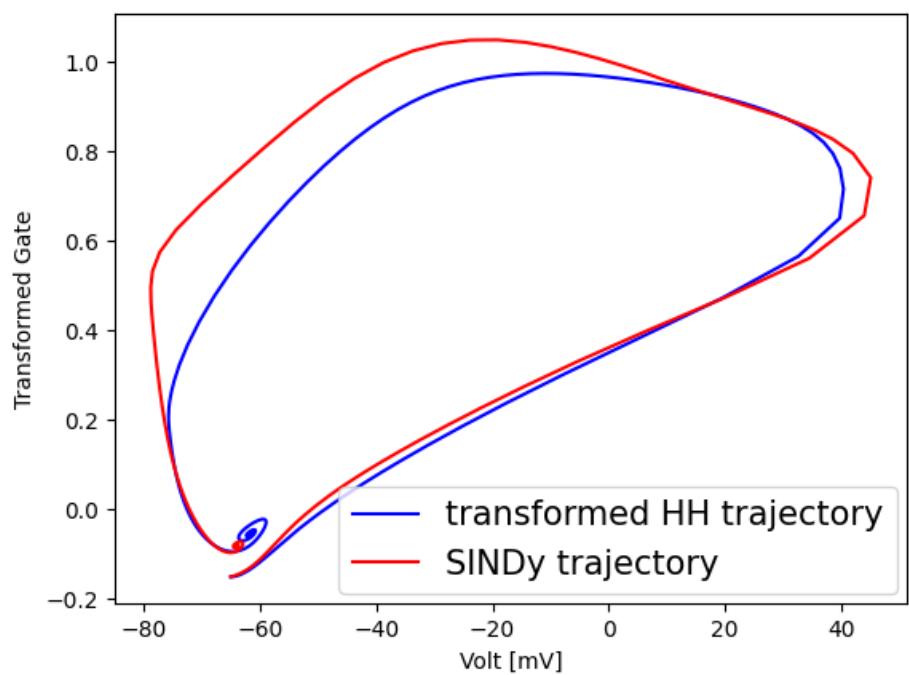
計算コストについての評価の結果を表 4.2 に示す。各項目の計算コストは、陽的 Euler 法を用いたシミュレーションで計測した。「浮動小数点演算数」と「指数演算数」は、タイムステップの計算に必要な浮動小数点演算の数と指数演算の数を表している。「空間計算量」は、シミュレーションの際に必要な浮動小数点を格納するメモリの数を表している。Hodgkin-Huxley モデルの場合、 V, m, h, n で 4 つ、代理モデルの場合、 V, g' で 2 つとなる。「実計算時間」は、再現性の評価に用いた設定と同じく、タイムステップ $dt = 0.05$ ms で 100 ms 間のシミュレーションを行った際にかかった実際の計算時間である。表 4.2 から代理モデルは、空間計算量は下がったが浮動小数点演算数や指数演算数などの時間計算量は上がっていることが分かる。

表 4.2: 各モデルが必要とする計算コスト

	浮動小数点演算数	指数演算数	空間計算量	実計算時間
Hodgkin-Huxley	174	6	4	0.036 ms
代理モデル	981	18	2	0.089 ms

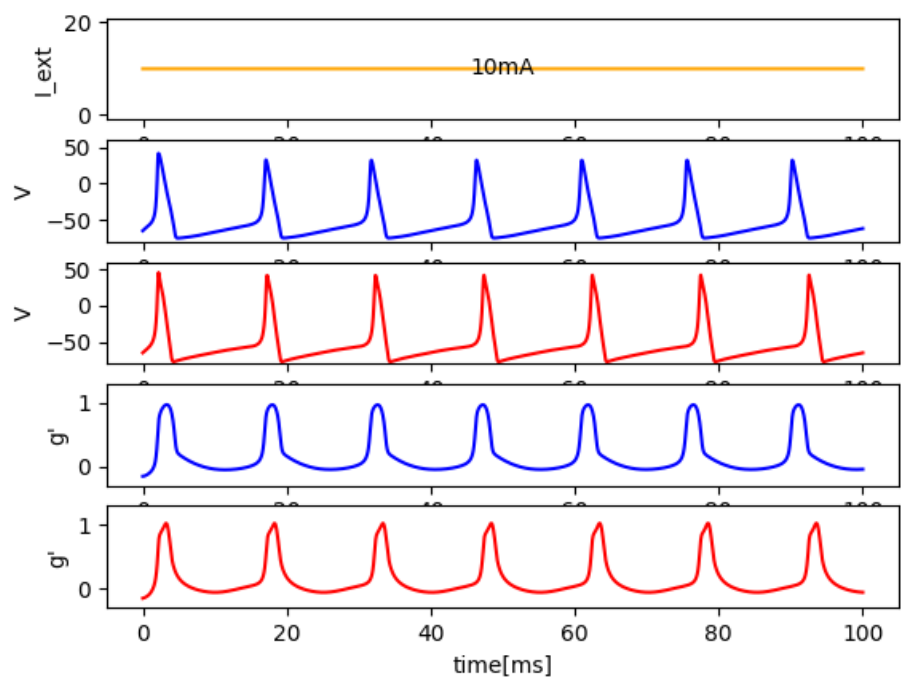


(a)

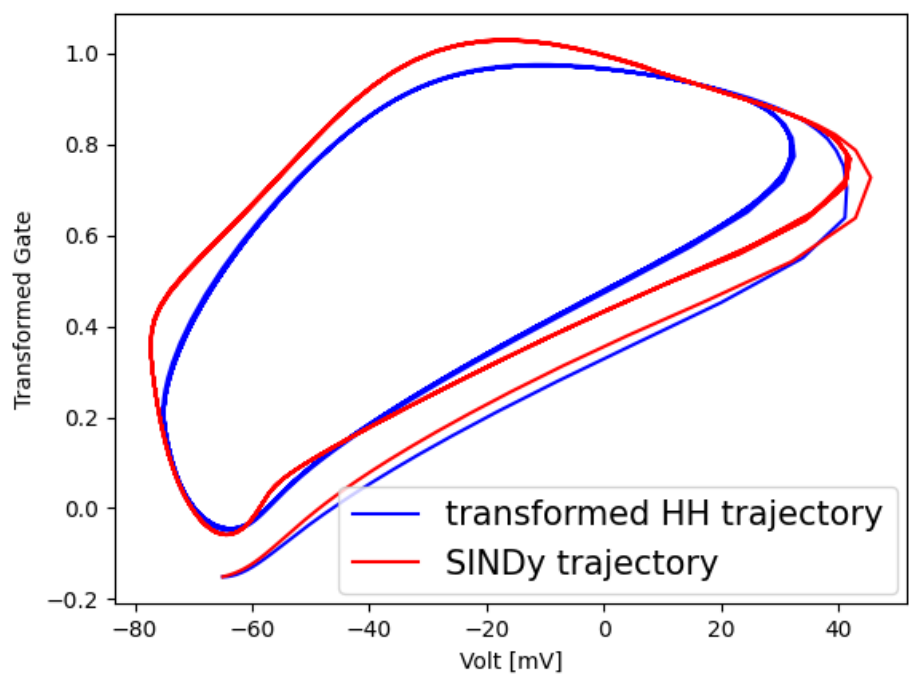


(b)

図 4.4: Hodgkin-Huxley モデルと代理モデルの応答の比較。上図 (a): 5 mA の定常電流を与えたシミュレーション結果。上から与えられれば外部電流 I_{ext} 、Hodgkin-Huxley モデルの膜電位 V 、主成分分析を使って 1 次元に圧縮した Hodgkin-Huxley モデルのゲート変数 g' 、代理モデルによる g' 下図 (b): V と g' の軌跡。黄: 加えられた外部電流、青: Hodgkin-Huxley モデルの結果、赤: 代理モデルの結果。

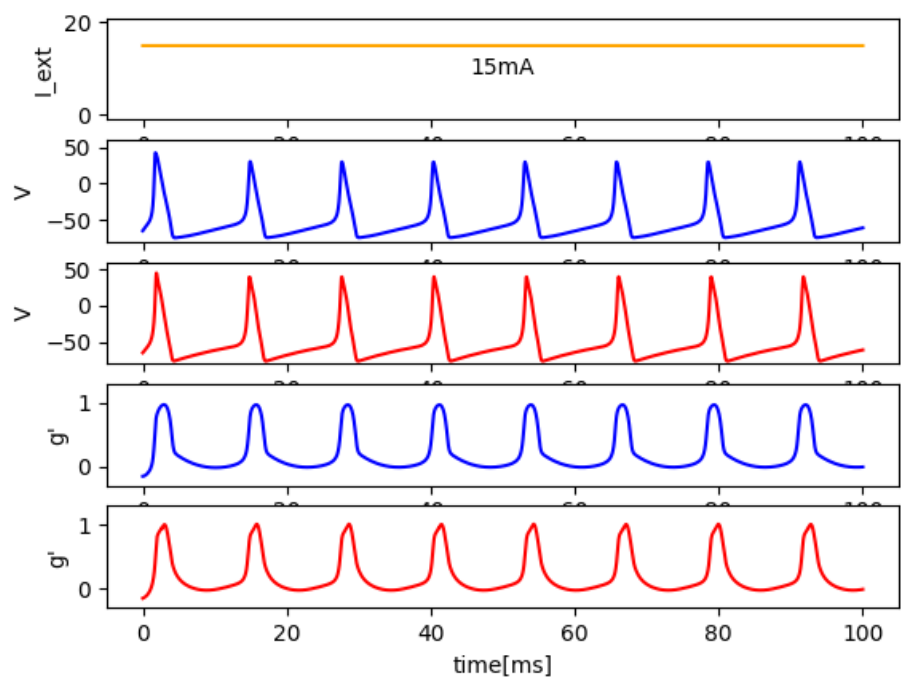


(a)

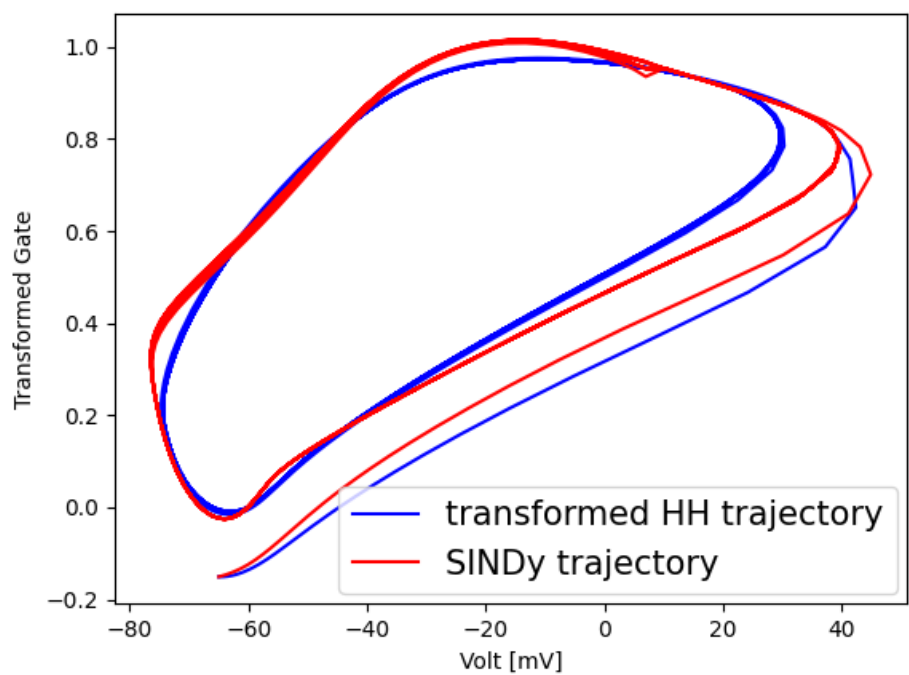


(b)

図 4.5: Hodgkin-Huxley モデルと代理モデルの応答の比較。上図 (a): 10 mA の定常電流を与えたシミュレーション結果。上から与えられれば外部電流 I_{ext} 、Hodgkin-Huxley モデルの膜電位 V 、主成分分析を使って 1 次元に圧縮した Hodgkin-Huxley モデルのゲート変数 g' 、代理モデルによる g' 下図 (b): V と g' の軌跡。黄: 加えられた外部電流、青: Hodgkin-Huxley モデルの結果、赤: 代理モデルの結果。

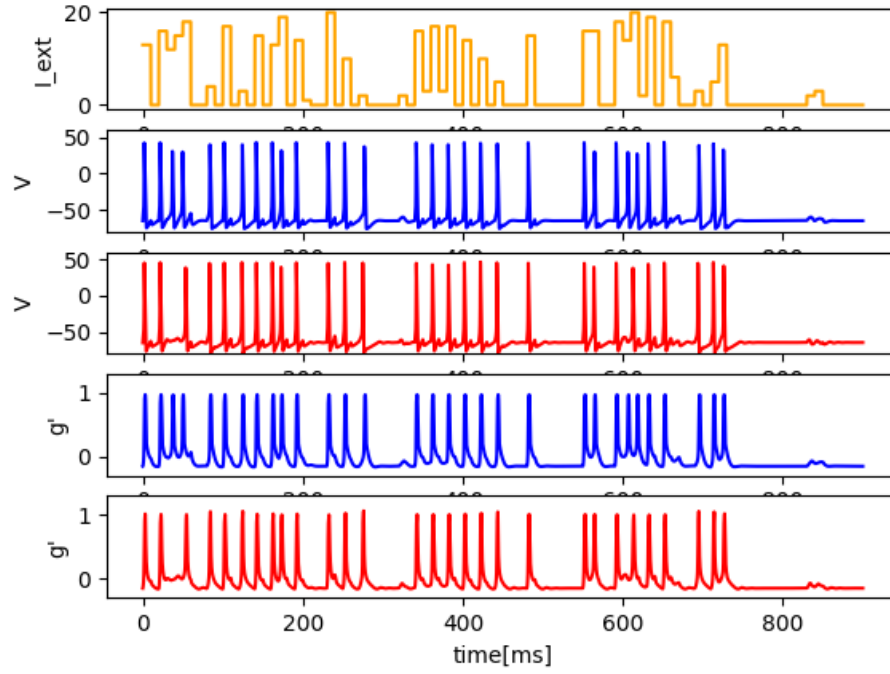


(a)

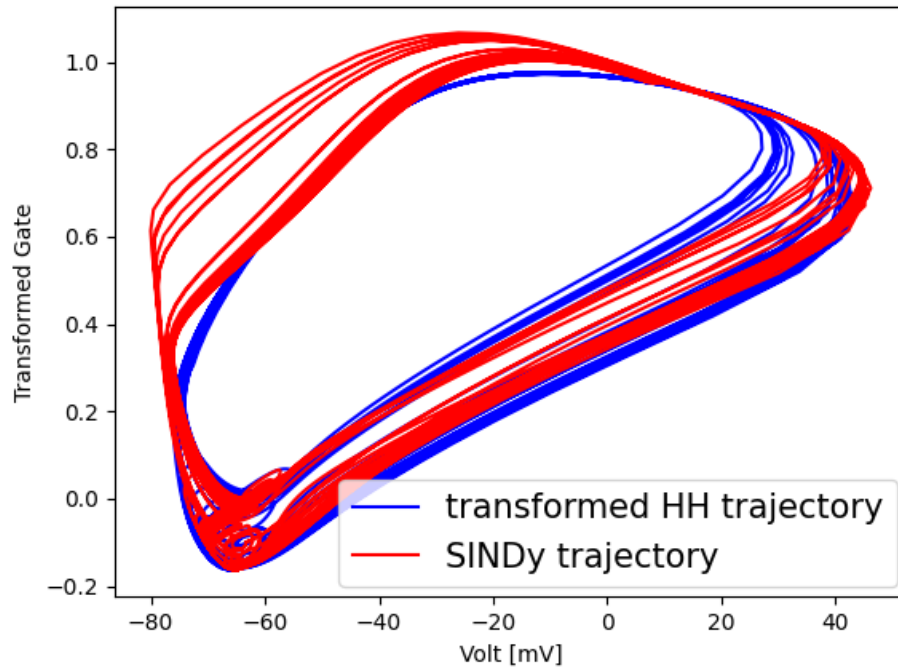


(b)

図 4.6: Hodgkin-Huxley モデルと代理モデルの応答の比較。上図 (a): 15 mA の定常電流を与えたシミュレーション結果。上から与えられれば外部電流 I_{ext} 、Hodgkin-Huxley モデルの膜電位 V 、主成分分析を使って 1 次元に圧縮した Hodgkin-Huxley モデルのゲート変数 g' 、代理モデルによる g' 下図 (b): V と g' の軌跡。黄: 加えられた外部電流、青: Hodgkin-Huxley モデルの結果、赤: 代理モデルの結果。



(a)



(b)

図 4.7: Hodgkin-Huxley モデルと代理モデルの応答の比較。上図 (a): 教師データの作成に用いた外部電流 I_{ext} を与えたシミュレーション結果。上から与えられれば外部電流 I_{ext} 、Hodgkin-Huxley モデルの膜電位 V 、主成分分析を使って 1 次元に圧縮した Hodgkin-Huxley モデルのゲート変数 g' 、代理モデルによる g' 下図 (b): V と g' の軌跡。黄: 加えられた外部電流、青: Hodgkin-Hyuxley モデルの結果、赤: 代理モデルの結果。

4.5 考察

代理モデルの膜電位応答は、元の Hodgkin-Huxley モデルと全く異なる形の式であるにも関わらず、非常に高い精度で再現できた。SINDy を用いた代理モデル作成は Zapadlo らなどによっても行われているが、比較すると膜電位とゲート変数の軌跡の再現性は本研究のほうが優れていた [15]。作成した代理モデルでの空間計算量は下がり、目的を達成することはできたが、時間計算量は上がった。これは、基底関数に対する係数が十分にスパースになっていないため (表 4.1)、代理モデルである式 4.1 の右辺の項数が抑えられていないことに起因する。基底関数に定数を掛けることで係数をスパースにするような工夫を行った (3.4.1 節) が、それだけでは不十分であることが分かる。SINDy 以外にも NeuralODE[16] のような代理モデル構築のためのアプローチを用いれば時間計算量についても低い代理モデルの構築も見込める。

本研究で用いた手法を Multi-Compartment モデルに対して適用させた場合、どれだけ空間計算量が削減されるのかを考えてみる。具体的には、コンパートメント数が 600 個、ゲート変数が 25 個、ニューロン共通の変数 (膜電位など) が 18 個である小脳の顆粒細胞の Multi-Compartment モデルを想定する [17]。そして、すべての変数は 8byte の double 型で格納すると考える。この場合、単一のニューロンの計算には、 $600 \times (25 + 18) \times 8 \approx 2 \times 10^6$ byte のメモリが必要となる。本研究で作成した代理モデルは、隠れ変数を m, h, n から g' の $\frac{1}{3}$ に抑えている。これが Multi-Compartment モデルにおいても適用されると仮定すると、先の Multi-Compartment モデルに対する代理モデルには、 $600 \times (25 \times \frac{1}{3} + 18) \times 8 \approx 1 \times 10^6$ byte のメモリが必要となり、空間計算量を半分ほど抑えられることが期待される。

現在、膜電位応答を再現しながら単一のニューロンの計算コストを下げる方法は主流ではない。例えば国吉らは、176,465 個ものニューロンによって構成された脳モデルのシミュレーションに Hodgkin-Huxley モデルよりも膜電位応答の再現性が劣っている積分発火モデルを使った [18]。今後は、必要な空間計算量をさらに下げることで、より実用に耐えうる代理モデルを作成することが必要である。

第 5 章 まとめ

本研究は、ニューロンモデルの一つである、Hodgkin-Huxley モデルの膜電位応答を再現する代理モデルの構築を行った。そのために、機械学習手法の一つである SINDy[11] や主成分分析を組み合わせた独自の手法 (3 章) を開発した。結果として、構築した代理モデルは、空間計算量を抑えながら Hodgkin-Huxley モデルの膜電位応答を高い精度で再現する能力を持った。このことから、シミュレーションにより多くの空間計算量を必要とする Multi-Compartment モデルに対する代理モデルの構築の見込みが立ったといえる。

今後は、Multi-Compartment モデルに対しての代理モデルの構築を行う必要がある。また、より使いやすくするために、代理モデルの時間計算量が増えてしまうことにも対処しなければならない。

謝辞

本研究を進めるにあたり、多大なるご指導とご助言を賜りました小林助教に深く感謝申し上げます。また、本研究は、データ解析およびモデル構築に際し、Python のオープンソースライブラリである `pysindy`, `mlflow` を使用しました。これらのライブラリの開発者およびオープンソースコミュニティの皆さまに深く感謝いたします。

参考文献

- [1] 山崎 匡 and 五十嵐 潤. はじめての神経回路シミュレーション:1 ニューロンからヒト全脳モデルまで. 森北出版株式会社, 2021 年 12 月 22 日, pp. 56–61.
- [2] Eric R. Kandel et al. カンデル神経科学. 第 2 版. メディカル・サイエンス・インターナショナル, 2022, p. 57.
- [3] Gidon Albert. *Dendritic Action Potentials and Computation in Human Layer 2/3 Cortical Neurons* / *Science*. <https://www.science.org/doi/10.1126/science.aax6239>. Jan. 2020. (Visited on 09/05/2024).
- [4] Tiago Branco, Beverley A. Clark, and Michael Häusser. “Dendritic Discrimination of Temporal Input Sequences in Cortical Neurons”. In: *Science (New York, N.Y.)* 329.5999 (Sept. 2010), p. 1671. DOI: [10.1126/science.1189664](https://doi.org/10.1126/science.1189664). (Visited on 09/05/2024).
- [5] Kaaya, Tamura Akira, and Rin Kuriyama. “Development of a lightweight and customizable biophysical neuron simulator”. 2024. URL: <https://researchmap.jp/tairakobayashi/presentations/48836360>.
- [6] A. L. Hodgkin and A. F. Huxley. “A Quantitative Description of Membrane Current and Its Application to Conduction and Excitation in Nerve”. In: *The Journal of Physiology* 117.4 (Aug. 1952), pp. 500–544. ISSN: 0022-3751. DOI: [10.1113/jphysiol.1952.sp004764](https://doi.org/10.1113/jphysiol.1952.sp004764).
- [7] 博義 宮川 and 雅司 井上. ニューロンの生物物理. 第 2 版. 丸善出版, 2013. (Visited on 08/27/2024).
- [8] 峯村吉泰. *C と Java で学ぶ数値シミュレーション入門*. 森北出版株式会社, 2008, pp. 155–157.
- [9] 瀧雅人. *機械学習スタートアップシリーズ これならわかる深層学習入門*. 株式会社 講談社, 2022, pp. 132–136.
- [10] *Data-Driven Science and Engineering : Machine Learning, Dynamical Systems, and Control*. 2nd Edition, p. 276.
- [11] Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. *Discovering Governing Equations from Data: Sparse Identification of Nonlinear Dynamical Systems*.

- <https://arxiv.org/abs/1509.03580v1>. Sept. 2015. DOI: [10 . 1073 / pnas . 1517384113](https://doi.org/10.1073/pnas.1517384113). (Visited on 09/05/2024).
- [12] Alan Kaptanoglu et al. “PySINDy: A Comprehensive Python Package for Robust Sparse System Identification”. In: *Journal of Open Source Software* 7.69 (Jan. 2022), p. 3994. ISSN: 2475-9066. DOI: [10.21105/joss.03994](https://doi.org/10.21105/joss.03994). (Visited on 09/18/2024).
 - [13] Brian De Silva et al. “PySINDy: A Python Package for the Sparse Identification of Non-linear Dynamical Systems from Data”. In: *Journal of Open Source Software* 5.49 (May 2020), p. 2104. ISSN: 2475-9066. DOI: [10.21105/joss.02104](https://doi.org/10.21105/joss.02104). (Visited on 09/18/2024).
 - [14] *STLSQ とは何か*. 2024. URL: <https://freshrimpsushi.github.io/jp/posts/2579/#fn:4>.
 - [15] Štěpán Zapadlo. “DSTLS SINDy Optimizer and Regression of Hodgkin-Huxley Model onto FitzHugh-Nagumo Model”. In: ().
 - [16] Ricky T. Q. Chen et al. *Neural Ordinary Differential Equations*. Dec. 2019. DOI: [10 . 48550/arXiv.1806.07366](https://doi.org/10.48550/arXiv.1806.07366). arXiv: [1806.07366](https://arxiv.org/abs/1806.07366) [cs]. (Visited on 02/04/2025).
 - [17] Katarzyna Dover et al. “FHF-independent Conduction of Action Potentials along the Leak-Resistant Cerebellar Granule Cell Axon”. In: *Nature Communications* 7.1 (Sept. 2016), p. 12895. ISSN: 2041-1723. DOI: [10.1038/ncomms12895](https://doi.org/10.1038/ncomms12895). (Visited on 02/06/2025).
 - [18] Yusuke Kuniyoshi et al. “Embodied Bidirectional Simulation of a Spiking Cortico-Basal Ganglia-Cerebellar-Thalamic Brain Model and a Mouse Musculoskeletal Body Model Distributed across Computers Including the Supercomputer Fugaku”. In: *Frontiers in Neurorobotics* 17 (Oct. 2023). ISSN: 1662-5218. DOI: [10 . 3389 / fnbot . 2023 . 1269848](https://doi.org/10.3389/fnbot.2023.1269848). (Visited on 05/09/2024).